Step 4. For $k = 1, \ldots, m$:

$$y_in_k = w_{0k} + \sum_{j=1}^{p} z_j w_{jk};$$

$$y_k = f(y_in_k).$$

6.1.3 Applications

Simple examples

The simple examples given here illustrate the training of a 2–4–1 backprop net (a net with two input units, four hidden units in one hidden layer, and one output unit) to solve the XOR problem. Example 6.1 uses binary data representation for this problem, with the binary sigmoid for the activation function on all hidden and output units. Example 6.2 uses bipolar data representation and the bipolar sigmoid function.

In each of these examples, the same set of initial weights is used; random values were chosen between -0.5 and $+0.5$.

The initial weights to the hidden layer are:

−0.3378	0.2771	0.2859	−0.3329	(biases to the four hidden units)
0.1970	0.3191	−0.1448	0.3594	(weights from the first input unit)
0.3099	0.1904	−0.0347	−0.4861	(weights from the second input unit).

The initial weights from the hidden units to the output unit are:

−0.1401	(bias on the output unit)
0.4919	(weight from the first hidden unit)
−0.2913	(weight from the second hidden unit)
−0.3979	(weight from the third hidden unit)
0.3581	(weight from the fourth hidden unit)

Although the training speed varies somewhat for different choices of the initial weights, the relative speeds shown here for these variations are typical. The learning rate for each of the examples is 0.02. Training continued until the total squared error for the four training patterns was less than 0.05.

Example 6.1 A backprop net for the XOR function: binary representation

Training using binary input is relatively slow; any unit that receives an input of zero for a particular pattern cannot learn that pattern. For the binary case, training took almost 3,000 epochs (see Figure 6.4).

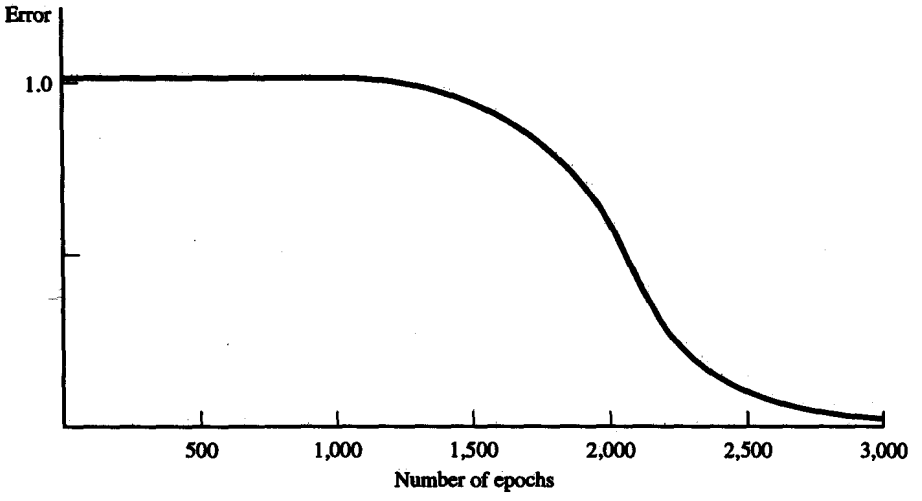


Figure 6.4 Total squared error for binary representation of XOR problem.

Example 6.2 A backprop net for the XOR function: bipolar representation

Using bipolar representation for the training data for the XOR problem and the bipolar sigmoid for the activation function for the hidden and output units gives much faster training than for the binary case illustrated in Example 6.1. Now, training takes only 387 epochs (see Figure 6.5).

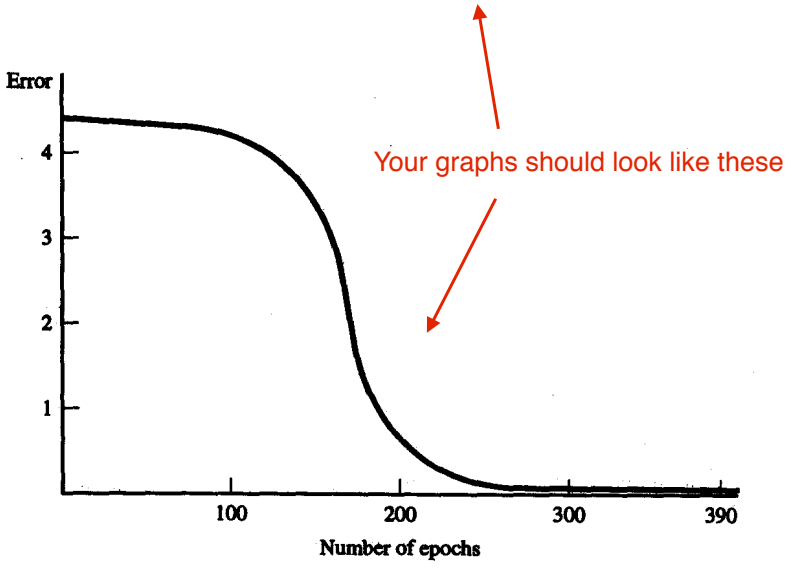


Figure 6.5 Total squared error for bipolar representation of XOR problem.

Example 6.3 A backprop net for the XOR function: modified bipolar representation

Convergence is often improved for target values that are not at the asymptotes of the sigmoid function, since those values can never be reached. Using the same initial weights and bipolar training input vectors as before, but targets of $+0.8$ or -0.8, gives convergence (total squared error $<.05$) in 264 epochs of training.

Example 6.4 A backprop net for the XOR function: Nguyen-Widrow weight initialization

Using Nguyen-Widrow weight initialization improved the training performance of the nets considered in Examples 6.1–6.3 (binary, standard bipolar, and bipolar with targets of $+0.8$ and -0.8). The weights from the hidden units to the output units were the same in each case. The Nguyen-Widrow initialization for the weights to the hidden units started with the values used in the previous examples. The weights were scaled so that the weight vector for each hidden unit was of length $0.7\sqrt{4} = 1.4$. The biases were scaled so that they fell between -1.4 and 1.4 (rather than between -0.5 and 0.5). The epochs required were as follows:

	RANDOM	NGUYEN-WIDROW
Binary XOR	2,891	1,935
Bipolar XOR	387	224
Modified bipolar XOR (targets = $+0.8$ and -0.8)	264	127

In these examples, the use of Nguyen-Widrow initialization not only improved the training speed, but also greatly reduced the chances of generating initial weights for which the net fails to converge. To study the sensitivity of these simple backpropagation nets to the choice of initial weights, eight different sets of random weights were used in the examples. For the binary representation, with random initial weights, only one of the eight cases converged in fewer than 3,000 epochs. Using the Nguyen-Widrow modification resulted in only one of the eight cases failing to converge; for the other seven cases, the training times ranged from 1,605 to 2,556 epochs.

In Examples 6.2 and 6.3, four of the weight sets caused the net to freeze (with all weights and biases equal to zero). With Nguyen-Widrow initialization, both forms of the bipolar XOR problem converged for all eight of the modified weights sets. For the standard bipolar data (Example 6.2), the training times ranged from 224 to 285 epochs. For the modified bipolar data (Example 6.3), with targets of $+0.8$ and -0.8, the training times ranged from 120 to 174 epochs.

Data compression

Example 6.5 A backprop net for data compression

Backpropagation can be used to compress data by training a net to function as an autoassociative net (the training input vector and target output vector are the same) with fewer hidden units than there are input or output units [Cottrell, Munro, & Zipser, 1989].

6.2 VARIATIONS

Several modifications can be made to the backpropagation algorithm presented on page 294 which may improve its performance in some situations. The modifications we discuss involve changes to the weight update procedure, alternatives to the sigmoid activation functions presented previously, a variation to improve biological plausibility and computational power, and finally an explicit statement of the backpropagation algorithm for 2 hidden layers.

6.2.1 Alternative Weight Update Procedures

Momentum

In backpropagation with momentum, the weight change is in a direction that is a combination of the current gradient and the previous gradient. This is a modification of gradient descent whose advantages arise chiefly when some training data are very different from the majority of the data (and possibly even incorrect). It is desirable to use a small learning rate to avoid a major disruption of the direction of learning when a very unusual pair of training patterns is presented. However, it is also preferable to maintain training at a fairly rapid pace as long as the training data are relative similar.

Convergence is sometimes faster if a momentum term is added to the weight update formulas. In order to use momentum, weights (or weight updates) from one or more previous training patterns must be saved. For example, in the simplest form of backpropagation with momentum, the new weights for training step $t + 1$ are based on the weights at training steps t and $t - 1$. The weight update formulas for backpropagation with momentum are

$$w_{jk}(t + 1) = w_{jk}(t) + \alpha\delta_k z_j + \mu[w_{jk}(t) - w_{jk}(t - 1)],$$

or

$$\Delta w_{jk}(t + 1) = \alpha\delta_k z_j + \mu\Delta w_{jk}(t),$$

and

$$v_{ij}(t + 1) = v_{ij}(t) + \alpha\delta_j x_i + \mu[v_{ij}(t) - v_{ij}(t - 1)],$$

or

$$\Delta v_{ij}(t + 1) = \alpha\delta_j x_i + \mu\Delta v_{ij}(t),$$

where the momentum parameter μ is constrained to be in the range from 0 to 1, exclusive of the end points.

Momentum allows the net to make reasonably large weight adjustments as long as the corrections are in the same general direction for several patterns, while using a smaller learning rate to prevent a large response to the error from any

one training pattern. It also reduces the likelihood that the net will find weights that are a local, but not global, minimum. When using momentum, the net is proceeding not in the direction of the gradient, but in the direction of a combination of the current gradient and the previous direction of weight correction.

As in the case of delta-bar-delta updates, momentum forms an exponentially weighted sum (with μ as the base and time as the exponent) of the past and present weight changes. Limitations to the effectiveness of momentum include the fact that the learning rate places an upper limit on the amount by which a weight can be changed and the fact that momentum can cause the weight to be changed in a direction that would increase the error [Jacobs, 1988].

Example 6.6 A backprop net with momentum for the XOR function: bipolar representation

Using the same initial weights and architecture as in Examples 6.1–6.3 and target values of +1.0 and −1.0, adding momentum (0.9) with a learning rate as before (0.2) reduces the training requirements from 387 epochs to 38 epochs.

Batch updating of weights

In some cases it is advantageous to accumulate the weight correction terms for several patterns (or even an entire epoch if there are not too many patterns) and make a single weight adjustment (equal to the average of the weight correction terms) for each weight rather than updating the weights after each pattern is presented. This procedure has a smoothing effect on the correction terms. In some cases, this smoothing may increase the chances of convergence to a local minimum.

Adaptive learning rates

The standard backpropagation algorithm modifies weights in the direction of most rapid decrease of the error surface for the current weights. In general, this does not move the weights directly toward the optimal weight vector. For further elaboration, see a standard work on adaptive filter theory, such as Widrow and Stearns (1985). A variety of methods to modify the direction of the weight adjustment have been proposed and studied.

One way in which researchers have attempted to improve the speed of training for backpropagation is by changing the learning rate during training. Some adjustable learning rate algorithms are designed for specific problems, such as classification problems in which there are significantly fewer training patterns from some classes than from others. If the traditional approach, duplication or creating noisy copies of the training patterns from the underrepresented classes, is not practical, the learning rate may be increased when training patterns from the underrepresented classes are presented [DeRouin, Brown, Beck, Fausett, & Schneider, 1991].